

Tópicos em IA / PLN: Criação de LLMs do Zero

Atividade Prática 1

1. Carregar uma amostra do *dataset* de teses e dissertações. Mostrar as informações básicas do *dataset*, por exemplo: número de documentos, número médio de parágrafos, número médio de palavras por documento. Gerar um arquivo txt, contendo todos os documentos. (sugestão: usar um delimitador para separar os documentos).
2. Instalar o tokenizador *Byte Pair Encoding (BPE)*, usado no treinamento de LLMs da família GPT (*GPT-like*).
 - Biblioteca *Python open source*: *tiktoken* (sugestão: `!pip install tiktoken`)
 - Carregar o esquema de tokenização usado no modelo GPT-2
 - Verificar se o tamanho do vocabulário é 50.257 tokens
 - Usar os métodos *encode()* e *decode()* para codificar/decodificar pequenos textos. Veja se “Raimundo Moura” gera os tokens: [49, 1385, 41204, 49902, 403]. Quais são os tokens gerados? Testar livremente outros três textos.
3. Gerar *pares input-target* (janelas deslizantes) para uma amostra de dados do arquivo txt (ou trecho inicial do arquivo).
 - Instalar o PyTorch (sugestão: `!pip install torch`)
 - Fazer iteração sobre o arquivo txt de entrada (*dataset*) e retornar as entradas (*inputs*) e alvos (*targets*) como *PyTorch tensors* (arrays multidimensionais)
 - Criar a classe *Dataset customizada* para definir como os registros de dados (textos) são carregados. Obs: considerar o arquivo txt.
 - Criar a classe *DataLoader* para embaralhar e montar os registros de dados (textos) em *batches*.
 - Testar livremente os parâmetros: *max_length*, *stride* e *batch_size*.
4. Criar a camada de *tokens embeddings* usando a função: *torch.nn.Embedding(vocab_size, output_dim)*. Para facilitar o entendimento, usar *vocab_size=50257* e *output_dim=256*.
 - Imprimir os *tokens embeddings* gerados para todo o vocabulário. (sugestão: usar o atributo *.weight* do objeto *embedding* criado)
 - Gerar os *tokens embeddings* para os *token ids* de entrada “Raimundo Moura”
5. Criar uma função para tratar os *embeddings* posicionais (*Absolute positional embedding*)
 - Usar valores sequenciais para os *embeddings* posicionais: 1.1, 1.2, 1.3, ...; 2.1, 2.2, 2.3, ...; ...6.1, 6.2, 6.3, ... (OBS: considerando a janela de contexto com 6 tokens, i.e, *max_length=6*)
 - Somar os *embeddings* posicionais com os *tokens embeddings* para obter os *input embeddings*.